

SIsomap: Secure Collaborative Manifold Learning with Reducing Communication Costs

Peizhao Zhou
zhoupz@mail.nankai.edu.cn
Nankai University
Tianjin, China

Ranyang Liu
liury@mail.nankai.edu.cn
Nankai University
Tianjin, China

Xiaojie Guo
xiaojie.veda@gmail.com
Shanghai Qi Zhi Institute
Shanghai, China

Lihai Nie*
Tong Li*
NLH@nankai.edu.cn
tongli@nankai.edu.cn
Nankai University
Tianjin, China

Pinzhi Chen
chenpinzhi@mail.nankai.edu.cn
Nankai University
Tianjin, China

Zheli Liu
liuzheli@nankai.edu.cn
Nankai University
Tianjin, China

Abstract

Secure manifold learning on datasets distributed among multiple data owners can benefit or even spawn many applications. For example, multiple service providers can jointly fit low-dimensional embeddings of their users' network behavior data to improve the accuracy of anomaly detection while addressing their privacy concerns about the datasets. In this paper, we focus on a classic manifold learning technique, known as isometric mapping (Isomap), and propose SIsomap, the first secure, distributed manifold learning system. We construct SIsomap based on secret sharing techniques and introduce careful optimizations. In particular, we propose two communication-efficient secure building blocks that focus on top- k and all-pairs shortest paths computation, respectively, and reduce secure operations by leveraging the characteristics of Isomap. Experimental results on both synthetic and real-world datasets demonstrate that our secure top- k and all-pairs shortest paths protocols are respectively up to $13.6\times$ and $1818.5\times$ faster than the state-of-the-art methods, and SIsomap as a whole is $11.1\times$ to $28.8\times$ faster than the baseline solution.

CCS Concepts

• Security and privacy → Privacy-preserving protocols.

Keywords

Isometric mapping, shortest paths, top- k , secret sharing

ACM Reference Format:

Peizhao Zhou, Xiaojie Guo, Pinzhi Chen, Ranyang Liu, Lihai Nie, Tong Li, and Zheli Liu. 2026. SIsomap: Secure Collaborative Manifold Learning with Reducing Communication Costs. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792105>

*Tong Li and Liha Nie are corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792105>

1 Introduction

Leveraging data collected from multiple entities, such as various companies or government agencies, often enhances the effectiveness of data mining. Due to legal restrictions or commercial competition, different entities may be reluctant to disclose their raw data directly. To address the privacy concerns of distributed data owners, numerous studies have developed secure solutions based on cryptographic techniques such as secret sharing, garbled circuits, and homomorphic encryption, all of which offer provable security. They facilitate joint data mining, including dimensionality reduction (focus on principal component analysis) [17, 24], classification [23, 28, 30], clustering [33, 46], regression [10, 32, 47], neural networks [29, 37, 45], etc.

However, manifold learning, a class of well-known data mining techniques widely used for nonlinear dimensionality reduction, denoising, and data visualization, has not been thoroughly studied in the context of privacy preservation. In this paper, we focus on a classical manifold learning algorithm, namely, isometric mapping (Isomap). Isomap seeks a lower-dimensional embedding that maintains geodesic distances between all points. Unlike traditional linear methods such as principal component analysis (PCA), Isomap [42] and its variants [12, 38, 41] are adept at capturing the nonlinear structures inherent in the data. This capability is crucial in fields where data exhibit intrinsic nonlinear relationships, such as network traffic analysis and bioinformatics [31].

Although research on secure Isomap is limited, several works come relatively close. PrivateGraph [40] develops graph spectral analysis protocols based on additive homomorphic encryption (AHE) and somewhat homomorphic encryption (SHE). It assumes the existence of a neighbor matrix or Laplacian matrix as input and constructs approximate eigen decomposition protocols to facilitate graph spectral analysis.

Although PrivateGraph is described as a cloud-based outsourcing model, it assumes the presence of a trusted party alongside the untrusted cloud server. This trusted party is responsible for generating and distributing public and private keys, as well as assisting the cloud server with computations. PPCA [17] constructs a privacy-preserving PCA using the secret-sharing-based secure multi-party computation (MPC) protocols. It employs the parallel Jacobi eigen

method and batch sorting technique to accelerate the computation. Subsequently, [24] also proposed a privacy-preserving PCA based on SHE. These works [17, 24, 40] are challenging to extend to Isomap due to the lack of efficient neighboring protocol and shortest path protocol.

In short, there is currently no work that supports secure manifold learning without assuming the existence of a precomputed intermediate state. In this paper, we propose SIsomap to fill this gap based on efficient MPC schemes. SIsomap benefits many privacy-preserving applications. For example, consider the following use case where multiple service providers each hold their own network behavior data of users. They wish to jointly compute a low-dimensional embedding across all datasets to identify global anomalous traffic, suspicious groups, or potential security threats, thereby facilitating cross-organizational threat detection and improving the accuracy of anomaly detection, without exposing their private data. The encrypted embedding can also be utilized for subsequent secure tasks, such as building anomaly detection models with SVMs or neural networks. We summarize our contributions as follows.

- To the best of our knowledge, SIsomap is the first protocol capable of performing complete manifold learning without assuming any precomputed intermediate states, such as a neighbor matrix or Laplacian matrix. We introduce careful optimizations, including reducing MPC operations and designing efficient protocols for top- k and all-pairs shortest paths calculation.
- Based on the quickselect algorithm, we propose a secure top- k protocol that achieves a linear number of secure comparisons with respect to input size. We also improve efficiency by introducing a new stopping condition that doubles the probability of algorithm completion after each partitioning step.
- We propose a round-efficient Floyd-Warshall protocol that serves to compute the all-pairs shortest paths. We improve communication batching by unrolling the inner two loops of the original algorithm and analyzing its data dependencies, thereby reducing round complexity. We also eliminate certain MPC operations based on the characteristics of Isomap. The round complexity of previous solutions is n times that of our protocol, where n is the number of vertices.
- We conduct comprehensive experiments. The results show that our secure top- k and all-pairs shortest paths protocols are respectively up to 13.6 $\times$ and 1818.5 $\times$ faster than the SOTA schemes, and SIsomap is 11.1 $\times$ to 28.8 $\times$ faster than the baseline solution.

2 SIsomap Overview

2.1 Architecture

SIsomap operates in an MPC-based outsourcing model and involves two roles: data owners and computing parties. SIsomap involves three computing parties and does not impose restrictions on the number of data owners. The computing parties can be cloud service providers such as Amazon Web Services and Alibaba Cloud, or they can be the data owners themselves. This is common in previous works [15, 21, 48]. The computing parties are connected through secure channels, for example, using OpenSSL.

As shown in Figure 1, SIsomap begins by data owners secretly sharing their samples with computing parties. Computing parties collaboratively perform Isomap computation based on the MPC

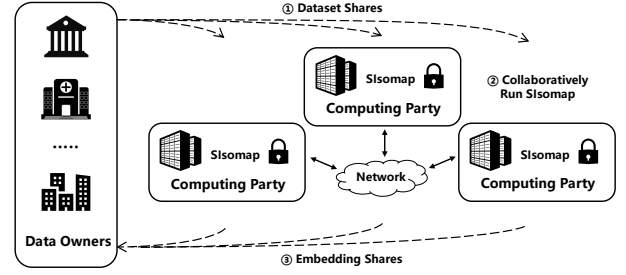


Figure 1: System architecture of SIsomap.

protocol and obtain secret shares of the low-dimensional embedding of the dataset, which are then sent back to some data owners who are prespecified to get the result. Data owners can reveal the clear embedding locally.

2.2 Security Model and Guarantees

SIsomap protects private data throughout the computation lifecycle. Overall, SIsomap protects data privacy using non-colluding computing parties, authenticated network links, and end-to-end oblivious computation. Aside from data size and neighboring parameters (the number of k -nearest neighbors and the radius), we ensure that no private datasets or intermediate data are leaked to a semi-honest adversary during evaluation. The semi-honest adversary passively eavesdrops on all internal states of one of the parties (e.g., memory contents, access patterns, and data sent/received) and attempts to infer private information. We formalize the security of SIsomap using the standard simulation-based paradigm [18] in Appendix B.

2.3 Technical Overview

Unlike traditional CPU instruction-based computation, communication overhead is considered the primary cost in MPC. In terms of arithmetic circuits, addition (similarly, subtraction) can be performed locally by the parties without communication, thus being regarded as free. Multiplication, comparison, and more complex functions like square roots and reciprocals require varying levels of interaction. Therefore, reducing the number of MPC operations is important for performance. Additionally, secret sharing scheme leads to communication rounds proportional to the circuit depth of the MPC operations. Reducing circuit depth and improving communication batching are also crucial for optimization.

Isomap consists of three components: neighboring computation, all-pairs shortest paths computation, and multidimensional scaling. Here, we give our high-level idea of how to efficiently realize them.

2.3.1 Neighboring computation. Isomap computes the k -nearest neighbors or radius neighbors for all samples.

k -nearest neighbors. Several studies provide methods for privately finding the k -nearest neighbors of a query point within a dataset. While it's possible to compute the k NN for each sample individually using these schemes, we avoid this approach for two reasons: (1) it results in unnecessary distance computations, and (2) these k NN schemes suffer from poor efficiency [27, 49], information leakage [39, 43], inaccuracy [11, 39], inflexibility for multiple data owners [11, 39], or high preprocessing costs [11, 27].

We first compute the distances between all pairs of samples and then find the top- k distances for all samples. Such that, we can

Table 1: The scale of secure operations in different secure top- k protocols.

Protocols	Number of Secure Comparisons	Number of Elements to Securely Shuffle
Exact protocol in [11]	$O(nk)$	None
Approximate protocol in [11]	$O(n + kb)$	None ²
Protocol in [35]	$O(n \log^2 k)$	None
This paper	$O(n)$	$O(n)$

¹ n is the input size, b is the number of buckets in [11].

² The approximate protocol in [20] requires local shuffling of n elements.

leverage the symmetry of the distance matrix to reduce the distance computations (corresponding to some MPC operations) by half

In this work, we propose a secure top- k protocol based on the quickselect algorithm. Our protocol is inspired by the quicksort protocol presented in [20], but employs a different skill for enhanced security. Since the output vector of quickselect is inherently uncertain, a straightforward extension from [20] is infeasible to achieve provable security as in [20]. To address this issue, instead of performing a one-time shuffling of the input vector as in [20], our approach shuffles the elements within the current partitioning range before each partitioning step. This modification enables us to establish provable security. We also introduce a novel stopping condition tailored for top- k selection, which doubles the probability of algorithm completion after each partitioning step.

We compare various secure top- k protocols in Table 1. Since these protocols employ different MPC techniques, it is more appropriate to compare the scale of their secure operations rather than their communication costs. Our proposed protocol requires fewer secure comparisons than existing methods, and the number of comparisons is independent of the parameter k . While our protocol involves securely shuffling $O(n)$ elements, advanced secure shuffle protocols incur communication costs linear with the number of elements [5, 16, 26], and thus do not affect the asymptotic complexity.

In the three-party setting, the secure radix sort proposed in [5] is actually more practical than the above protocols tailored for top- k selection. Radix sort needs about $O(\ell n \log n + \ell n)$ bits of interactions, while our protocol exhibits less interactions of $O((\ell + \log n)n)$ bits.

Radius neighbors. We first compute the distances between all pairs of samples and then set the distances greater than the radius to a tremendous value via secure comparators and multiplexers. Given the symmetry of the distance matrix, we ensure that all MPC operations between any pair of samples are performed only once, thereby enhancing performance.

2.3.2 All-pairs shortest paths computation. The most well-known algorithms for finding shortest paths are the Dijkstra algorithm and the Floyd-Warshall algorithm. In the context of privacy preservation, secure versions of the Dijkstra algorithm have been proposed [1, 2, 36]. However, we identified two drawbacks of the Dijkstra algorithm leading to high communication costs in its secure versions: (1) it requires random access to the data, and (2) it does not parallelize well and results in high rounds. For instance, the secure Dijkstra protocol in [1] needs $O(\ell n^3 \log n)$ bits of communication in $O(n^2 \log \ell)$ rounds to compute the all-pairs shortest paths for an n -vertex graph. In our test, it takes more than 1 hour on a 200-vertex graph when using a semi-honest three-party replicated

Table 2: The communication costs of all-pairs shortest paths calculation using different protocols.

Protocols	Communication (bits)	Rounds
Dijkstra-based [1]	$O(\ell n^3 \log n)$	$O(n^2 \log \ell)$
Plain Floyd-Warshall	$O(\ell n^3)$	$O(n^3 \log \ell)$
This paper	$O(\ell n^3)$	$O(n \log \ell)$

Notes: n is the number of vertices and ℓ is the bit length of distance.

secret sharing scheme. This contributes to the main bottleneck for the practical performance of secure Isomap.

We observed that the Floyd-Warshall algorithm, with its fixed data access pattern, may be more advantageous for consideration within an MPC setting. However, a significant challenge is that the algorithm's triple-loop structure results in a cubic round complexity relative to the number of vertices n , making it highly inefficient. In this work, we propose a round-efficient secure Floyd-Warshall protocol. At a high level, we begin by unrolling the inner two loops and analyzing the data dependencies within the computations, dividing the process into three distinct batches. Within each batch, interactions can be packaged together, substantially reducing the number of rounds. Moreover, we eliminate certain MPC operations based on Isomap's characteristics, further improving the performance. The costs of different protocols are detailed in Table 2.

2.3.3 Multidimensional scaling. Isomap applies multidimensional scaling on the distance matrix of the shortest paths to get the low-dimensional embedding of dataset. This process involves applying double centering to transform the squared distance matrix into a matrix of inner products, followed by eigen decomposition. When computing the squared distance matrix, we restrict multiplications to the upper triangular portion of the matrix. For eigen decomposition, we employ the parallel Jacobi method as described in PPCA [17], which has been demonstrated to be more efficient than the QR shift method in the MPC setting.

3 Preliminaries

Let $S \setminus w$ denote the removal of element w from the set S . We use G^T to denote the transpose of a matrix G . We use ∞ to indicate that two samples are not directly connected, which is typically replaced by a tremendous distance value. We use $x \parallel y$ to denote the bitwise concatenation of value x and value y , and $x^{[n,m]}$ to denote the bits of value x from the m -th bit to the n -th bit.

3.1 Three-Party Replicated Secret Sharing

We use the replicated secret sharing scheme [4]. This scheme divides number $x \in \{0, 1\}^\ell$ into three uniformly random parts $x = x_1 + x_2 + x_3 \pmod{2^\ell}$, with party 1 holds share (x_1, x_2) , party 2 holds share (x_2, x_3) , and party 3 holds share (x_3, x_1) . When the positive integer $\ell > 1$, the sharing type is arithmetic sharing; otherwise, it is Boolean sharing. Conversions between arithmetic share and Boolean share are done by evaluating a parallel prefix adder [32]. We use $\llbracket x \rrbracket$ to denote the secret sharing of x . Any two parties are sufficient to reveal a shared value by gathering the above three parts. The revealing protocol is denoted as $x \leftarrow \text{REVEAL}(\llbracket x \rrbracket)$.

The commutative nature of addition allows one to add or subtract secret-shared values without communication. Parties must interactively multiply $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$. Observe that, $xy = (x_1 + x_2 + x_3)(y_1 +$

$y_2 + y_3$), and each party can compute part of the cross-terms locally. That is, party i can locally compute z_i , for $z_1 := x_1y_1 + x_1y_2 + x_2y_1$, $z_2 := x_2y_2 + x_2y_3 + x_3y_2$, and $z_3 := x_3y_3 + x_3y_1 + x_1y_3$. Then, party i adds a zero sharing α_i to z_i for rerandomizing the shares where $\sum_i \alpha_i = 0$. Such a zero sharing can be computed without any interaction after a one-time setup. Finally, party i sends z_i to party $i - 1$ to satisfy the replication requirement.

Fixed-point representation. A fixed-point value is defined as an ℓ -bit integer using twos-complement where the bottom f bits denote the decimal part. This means that for positive numbers, bit i denotes the $(i - f)$ th power of 2. Addition and subtraction are done locally as with integer representation because the results are expected to remain below 2^ℓ . Multiplication can also be performed in the same manner, but must be additionally divided by 2^f to maintain the f decimal bits invariant since the number of decimal bits doubles. We realize this with the probabilistic truncation protocol of [14].

Matrix multiplication. Matrix multiplication can be implemented through a series of dot products. An important optimization is that the conversion from 3-out-of-3 shares to 2-out-of-3 shares and the truncation of each multiplication can be delayed after the summation in the dot product of integer representation [32].

Comparison and multiplexer. The comparisons between two shared values, denoted as $[[x]] < [[y]]$ etc., are reduced to computing the most significant bit of the difference between the two values, which can be done by evaluating a parallel prefix adder that has removed some unnecessary gates [32].

We denote a multiplexer as $[[v]] \leftarrow_{[[c]]} [[a]] : [[b]]$, where $v = a$ if $c = 1$, otherwise $v = b$. Multiplexer is implemented as $[[v]] \leftarrow [[b]] + [[c]] \langle [[a]] - [[b]] \rangle$. Specially, a minimum operation is defined as $\text{MIN}([a], [b]) =_{[[a]] < [[b]]} [[a]] : [[b]]$.

Square root. We use the numerical method based on Newton-Raphson and Goldschmidt iterations from [3] to calculate the square root of a secret-shared value. We denote the protocol as $[[y]] \leftarrow \text{SQRT}([x])$ where $y = \sqrt{x}$.

Set operations. A permutation π is a bijective function from a finite set to itself, usually the underlying set is $\{1, \dots, n\}$. We denote $\pi = (\pi_1, \dots, \pi_n)$ as the vector of destinations of the permutation π . Given vector x , we denote $y = \pi(x)$ as applying π on x which gives $y = (x_{\pi_1}, \dots, x_{\pi_n})$.

Recent work presents a secure shuffle protocol and a secure permutation protocol available for replicated secret sharing [5]. Both protocols are highly efficient, requiring only $O(n)$ bits of communication over $O(1)$ rounds to process an n -element secret-shared vector. The shuffle protocol securely applies a random permutation to a shared vector, denoted as

$$[[b]] \leftarrow \text{SHUFFLE}([a])$$

where $b = \pi^r(a)$ and π^r is a random permutation.

Given a shared permutation $[[\pi]]$ and a shared vector $[[a]]$, the permutation protocol securely applies $[[\pi]]$ on $[[a]]$, denoted as

$$[[b]] \leftarrow \text{PERMUTE}([[\pi]], [a])$$

where $b = \pi(a)$. Note that $[[b]] \leftarrow \text{PERMUTE}([[\pi]], [a], \text{reverse})$ denotes reversely applying $[[\pi]]$ on $[[a]]$ where $b = \pi^{-1}(a)$.

Eigen decomposition. We use the parallel Jacobi method described in [17] to perform the eigen decomposition on secret-shared

matrix. We denote the protocol as $[[v]], [[V]] \leftarrow \text{JACOBI}([X])$. Given a secret-shared matrix X , the protocol returns all eigenvalues v and corresponding eigenvectors (columns) V with secret-shared.

4 Secure Isometric Mapping

This section details SIsomap by dividing it into three consecutive components: nearest neighbors computation, all-pairs shortest paths computation, and multidimensional scaling.

4.1 Nearest Neighbors Computation

Given the secret-shared samples from different owners, we first compute the neighbor matrix G that stores the k -nearest neighbors or radius neighbors for all samples. $G_{i,j}$ represents the distance from i -th sample to j -th sample. In this work, we use the Euclidean distance as metric, which is common in Isomap.

4.1.1 Top- k search. The key building block of k -nearest neighbors computation is the top- k search. In this section, we present a secure top- k search protocol based on the quickselect algorithm. Quickselect is a simplified variant of the quicksort algorithm, specifically designed to efficiently identify the k -th smallest or largest element in an array. Unlike quicksort, which recursively partitions both sides of a pivot, quickselect only concerns the side that contains the k -th element. Both algorithms are not oblivious by default, which means that implementing textbook algorithms using secure computation techniques is insecure. The work in [20] proposes a secure version of the quicksort. A natural idea is to extend their approach to realize a secure version of quickselect as below.

A protocol using the methods of [20]. The protocol assumes that the elements of the input vector are unique. Given an n -element secret-shared vector, the protocol performs the following steps.

- Step 1 *Elements uniquifying.* If the elements of the input vector cannot be guaranteed to be unique, appending a log n -bit value in $\{1, \dots, n\}$ to each element of the input vector.
- Step 2 *Vector shuffling.* Securely shuffle the input vector.
- Step 3 *Vector partitioning.* (1) Select a pivot and compare it with the remaining elements. (2) Reveal the secret-shared comparison results and partition the elements based on them.
- Step 4 Check whether the pivot is the k -th element. If so, return; otherwise, reduce the partitioning range to the side containing the k -th element and go back to Step 3.

The unique-element assumption and the vector shuffling are intended to ensure that the revealed comparison results and their distribution are independent of the actual input vector.

However, it is difficult to provide a provable security for this protocol. The simulation-based security proof of secure quicksort in [20] relies on the fact that the output of quicksort is deterministic if the input vector has unique elements, then the permutation between the shuffled vector and the sorted vector can be simulated by a random permutation. But this is not the case for quickselect, as the quickselect that incorporates a stochastic process has an uncertain output vector.

Our solution. To solve this problem, instead of shuffling the vector once at the beginning, we shuffle the elements within the partitioning range before each partitioning step.

In addition, we propose a more efficient stopping condition tailored for the top- k search. In a standard quickselect, the algorithm stops when the pivot index is k , as its goal is to find the k -th smallest/largest element. While this stopping condition also works for finding the top- k elements (i.e., the k -smallest/largest elements), we propose a better one: partitioning stops when the pivot index is k or $k + 1$. Our stopping condition works because: assuming that $\mathbf{a} = (a_1, \dots, a_n)$ is the input vector and we want to find the k -smallest elements, when a_{k+1} is the pivot, we have $a_i \leq a_{k+1} \leq a_j$, for any $i \in \{1, \dots, k\}$ and $j \in \{k + 2, \dots, n\}$, thus $a_i \leq a_m$, for any $m \in \{k + 1, \dots, n\}$, meaning that $(a_1, \dots, a_k)$ is also a valid top- k result. Since the index of the randomly picked pivot allows the algorithm to stop in either of two cases, the probability of the algorithm stopping doubles after each partition, and the average number of partitions is expected to be reduced.

The main steps of our secure top- k protocol are as follows.

- Step 1 *Elements uniquifying*. If the elements of the input vector cannot be guaranteed to be unique, appending a $\log n$ -bit value in $\{1, \dots, n\}$ to each element of the input vector.
- Step 2 *Vector shuffling and partitioning*. (1) **Securely shuffle the elements within the partitioning range**. (2) Select a pivot and compare it with the remaining elements. (3) Reveal the secret-shared comparison results and partition the elements based on them.
- Step 3 **Check whether the pivot is the k -th or $(k + 1)$ -th element**. If so, return; otherwise, reduce the partitioning range to the side containing the k -th and $(k + 1)$ -th element and go back to Step 2.

The detailed secure top- k protocol is presented in Protocol 1. Given secret-shared vector $\mathbf{a}$, the protocol aims to find the k -smallest elements in $\mathbf{a}$. It permutes the k -smallest elements to the first k positions in the vector and returns a permutation that maps the input vector to the output vector. We note that the appending bits used to unify the vector elements contribute exactly to this permutation.

The partitioning range (l, h) is initialized as $(1, n)$. After elements uniquifying, the protocol repeatedly shuffles and partitions the elements within a shrinking partitioning range until the stopping condition is met. When the k -smallest elements are found, we separate the appending bits from the vector elements and assign them as the permutation π .

In PARTITION, the protocol (randomly) picks a pivot, compares it with the remaining elements, and reveals the comparison results. Based on the public comparison results, parties locally swap the elements smaller than the pivot to its left side and those greater than the pivot to its right side.

Complexity. It has been proven in [13] that the average time complexity of quickselect is $O(n)$ and the average number of partitions is $O(\log n)$. The time complexity indicates that the number of secure comparisons and shuffled elements in Protocol 1 are both $O(n)$, corresponding to $O((\ell + \log n)n)$ bits of communication complexity. The number of partitions corresponds to a round complexity of $O(\log n \log \ell)$, since the communications of secure comparisons in each partition are performed in parallel.

Security. Intuitively, since we shuffle the elements within the partitioning range before each partition, the pivot and the remaining elements are selected randomly, so that their comparison results are

Protocol 1: Secure top- k

Notation: $[[z]], [[\pi]] \leftarrow \text{TOPK}([[\mathbf{a}]], k)$.

Input: Secret-shared vector $\mathbf{a} = \{a_i\}_{i=1}^n$ where $a_i \in \{0, 1\}^\ell$, parameter $1 \leq k \leq n$.

Output: Secret-shared vector $\mathbf{z}$ and permutation π .

```

1 Set  $[[\mathbf{u}]] = ([[1]], \dots, [[n]])$  where  $u_i \in \{0, 1\}^{\log n}$ .
2 Set  $[[\mathbf{b}]] = ([[a_1 || u_1]], \dots, [[a_n || u_n]])$ .
3  $l \leftarrow 1$  and  $h \leftarrow n$ .
4 while true do
5    $([[b_l]], \dots, [[b_h]]) \leftarrow \text{SHUFFLE}([[\mathbf{b}]], l, h)$ .
6    $p, [[\mathbf{b}]] \leftarrow \text{PARTITION}([[\mathbf{b}]], l, h)$ .
7   if  $p = k$  or  $p = k + 1$  then
8      $[[z]] =$ 
7        $([[b_1^{\ell + \log n, \log n + 1}]], \dots, [[b_n^{\ell + \log n, \log n + 1}]]);$ 
9      $[[\pi]] = ([[b_1^{\log n, 1}]], \dots, [[b_n^{\log n, 1}]]);$ 
10    return  $[[z]], [[\pi]]$ .
11  else if  $p < k$  then  $l \leftarrow p + 1$ ;
12  else  $h \leftarrow p - 1$ ;
```

Notation: $p, [[\mathbf{b}]] \leftarrow \text{PARTITION}([[\mathbf{b}]], l, h)$.

Input: Secret-shared vector $\mathbf{b} = \{b_i\}_{i=1}^n$ where $b_i \in \{0, 1\}^{\ell + \log n}$, the lower bound of the partition range l , and the upper bound of partition range h .

Output: Pivot index p , and partitioned shared vector $\mathbf{a}$.

```

13  $i \leftarrow l - 1$ .
14 for  $j = l$  to  $h - 1$ , in parallel :
15    $[[e_j]] \leftarrow ([[b_j]] < [[b_h]])$ .
16  $\mathbf{e} \leftarrow \text{REVEAL}([[\mathbf{e}]])$ .
17 for  $j = l$  to  $h - 1$ :
18   if  $e_j = 1$  then
19      $i \leftarrow i + 1$ .
20   Swap  $[[b_i]]$  and  $[[b_j]]$ .
21  $p \leftarrow i + 1$ .
22 Swap  $[[b_p]]$  and  $[[b_h]]$ .
23 return  $p, [[\mathbf{b}]]$ .
```

independent of the actual values. Appending the elements prevents the distribution of the comparison results from exposing information about the number of duplicate actual values. We formalize the security of Protocol 1 in Appendix B.

Note that, for the protocol using the methods of [20], the pivot and the remaining elements are not selected randomly starting from the second partitioning step because the elements have been moved in a non-random manner during the previous partitions.

We define the functionality of Protocol 1 as follows.

$\mathcal{F}_{\text{topk}}$: On input $[[\mathbf{a}]]$ and k from parties, the functionality reveals $\mathbf{a}$ and appends values in $\{1, \dots, n\}$ to each element of $\mathbf{a}$. It computes vector $\mathbf{z}$ and permutation π , such that $(z_1, \dots, z_n) = (a_{\pi_1}, \dots, a_{\pi_n})$ and $z_i \leq z_j$ for any $i \in \{1, \dots, k\}$, $j \in \{k + 1, \dots, n\}$. Finally, $\mathbf{z}$ and π are secretly shared with the parties.

4.1.2 k -Nearest Neighbors. On input a secret-shared input dataset, protocol 2 computes the k NN matrix $\mathbf{G}$. The protocol initializes $\mathbf{G}$ with zeros and computes the Euclidean distances between all

Protocol 2: Secure k -nearest neighbors**Notation:** $[[G]] \leftarrow \text{KNEIGHBORS}([X], k)$.**Input:** Secret-shared dataset $X \in \mathbb{R}^{n \times m}$ with n samples and m dimensions, the number of neighbors k .**Output:** Secret-shared neighbor matrix $G \in \mathbb{R}^{n \times n}$.

```

1 Initialize  $[[G]]$  as  $G = 0^{n \times n}$ .
2 for  $i = 1$  to  $n - 1$ , in parallel :
3   for  $j = i + 1$  to  $n$ , in parallel :
4      $[[G_{i,j}]] \leftarrow \text{SQRT}(\sum_{w=1}^m ([X_{i,w}] - [X_{j,w}])^2)$ .
5  $[[G]] \leftarrow [[G]] + [[G^T]]$ .
6 for  $i = 1$  to  $n$ , in parallel :
7    $[[G_i]], [[\pi]] \leftarrow \text{TOPK}([G_i], k)$ .
8   for  $j = k + 1$  to  $n$  :
9      $[[G_{i,j}]] \leftarrow [[\infty]]$ .
10   $[[G_i]] \leftarrow \text{PERMUTE}([[\pi]], [G_i], \text{reverse})$ .
    # Symmetrize the neighbor matrix.
11 for  $i = 1$  to  $n - 1$ , in parallel :
12   for  $j = i + 1$  to  $n$ , in parallel :
13      $[[d]] \leftarrow \text{MIN}([G_{i,j}], [G_{j,i}])$ .
14      $[[G_{i,j}]] \leftarrow [[d]]$  and  $[[G_{j,i}]] \leftarrow [[d]]$ .
15 return  $[[G]]$ .
```

Protocol 3: Secure radius neighbors**Notation:** $[[G]] \leftarrow \text{RNEIGHBORS}([X], d_{\text{radius}})$.**Input:** Secret-shared dataset $X \in \mathbb{R}^{n \times m}$ with n samples and m dimensions, and the radius d_{radius} .**Output:** Secret-shared neighbor matrix $G \in \mathbb{R}^{n \times n}$.

```

1 Initialize  $[[G]]$  as  $G = 0^{n \times n}$ .
2 for  $i = 1$  to  $n - 1$ , in parallel :
3   for  $j = i + 1$  to  $n$ , in parallel :
4      $[[d]] \leftarrow \text{SQRT}(\sum_{w=1}^m ([X_{i,w}] - [X_{j,w}])^2)$ .
5      $[[c]] \leftarrow [[d]] > d_{\text{radius}}$ .
6      $[[G_{i,j}]] \leftarrow [[c]] : [[\infty]] : [[d]]$ .
7  $[[G]] \leftarrow [[G]] + [[G^T]]$ .
8 return  $[[G]]$ .
```

pairs of samples. Given that the distance from i -th sample to j -th sample is equal to the distance from j -th sample to i -th sample, the protocol only computes the distances in the upper triangle of G . The protocol adds the matrix to its transpose to assign the value at position (i, j) to position (j, i) .

Then, for each row, the protocol arranges the k -nearest distances to the first k positions using the top- k protocol and sets the distances in the last $n - k$ positions to ∞ . Distances are rearranged to their original positions by reversely applying the corresponding permutation on each row. Finally, the k NN matrix G is symmetrized by setting the values at a given position and its transposed position to the minimum of the two.

4.1.3 Radius Neighbors. Protocol 3 computes the radius-neighbor matrix on a secret-shared input dataset. The radius d_{radius} is a user-defined parameter. The protocol initializes the neighbor matrix with zeros and computes the Euclidean distances for the positions within the upper triangle of the matrix. The protocol resets the

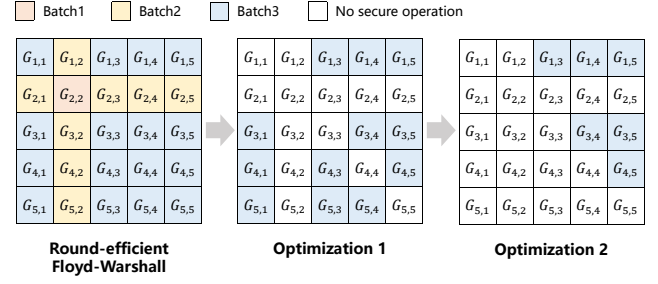


Figure 2: Diagram of Round-efficient Floyd-Warshall protocol and its further optimizations when $w = 2$.

distances greater than the radius to ∞ using a secure comparator and multiplexer. Finally, we add the neighbor matrix to its transpose to assign the values of the upper triangle to their corresponding transposed positions in the lower triangle.

4.2 All-Pairs Shortest Paths Computation

In this section, we propose an optimized protocol based on the Floyd-Warshall algorithm. On input matrix G representing an n -vertex graph, a textbook implementation of the Floyd-Warshall algorithm involves three nested loops.

for $w = 1$ to n :

for $i = 1$ to n :

for $j = 1$ to n :

$[[G_{i,j}]] \leftarrow \text{MIN}([G_{i,j}], [G_{i,w}] + [G_{w,j}])$.

This implementation updates the distance from vertex i to vertex j by considering an intermediate vertex w . However, it leads to a circuit depth of $O(n^3)$. Since the round complexity of a secret sharing protocol is proportional to the circuit depth, this is impractical.

Round-efficient Floyd-Warshall. We unroll the inner two loops of the Floyd-Warshall algorithm, analyze its data dependencies and perform the interactions of independent operations in a batch, thereby significantly reducing the round complexity. In detail, for each certain value of $w = w'$, we divide the operations into the following three batches.

- Batch1: Including the operation for $i = j = w'$, that is, $G_{w',w'} = \text{Min}(G_{w',w'}, G_{w',w'} + G_{w',w'})$.
- Batch2: (1) Including operations for $i = w', j \neq w'$, that is, for all $j \in \{1, \dots, n\} \setminus w'$, $G_{w',j} = \text{Min}(G_{w',j}, G_{w',w'} + G_{w',j})$.
(2) Including operations for $i \neq w', j = w'$, that is, for all $i \in \{1, \dots, n\} \setminus w'$, $G_{i,w'} = \text{Min}(G_{i,w'}, G_{i,w'} + G_{w',w'})$.
- Batch3: Including operations for $i, j \neq w'$, that is, for all $i, j \in \{1, \dots, n\} \setminus w'$, $G_{i,j} = \text{Min}(G_{i,j}, G_{i,w'} + G_{w',j})$.

OBSERVATION 1 (DATA DEPENDENCIES). (1) The operation in Batch1 does not depend on the computation results of other batches. (2) The operations in Batch2 only depend on the computation results of Batch1: $G_{w',w'}$. (3) The operations in Batch3 only depend on the computation results of Batch2: $G_{i,w'}$ and $G_{w',j}$.

Based on the data dependencies in Observation 1, the minimum operations in each batch can be performed in parallel as long as

Protocol 4: Secure Floyd-Warshall

Notation: $[[D]] \leftarrow \text{FLOYD}([G])$.
Input: Secret-shared neighbor matrix $G \in \mathbb{R}^{n \times n}$.
Output: Secret-shared distance matrix $D \in \mathbb{R}^{n \times n}$.

```

1 for  $w = 1$  to  $n$  :
    # Compute Batch3 in the upper triangle.
2   for  $i = 1$  to  $n - 1$ , in parallel :
3     for  $j = i + 1$  to  $n$ , in parallel :
4       if  $i \neq w$  and  $j \neq w$  then
5          $[[G_{i,j}]] \leftarrow$ 
            $\text{MIN}([G_{i,j}], [[G_{i,w}]] + [[G_{w,j}]])$ .
    # Update Batch3 in the lower triangle.
6   for  $i = 2$  to  $n$ , in parallel :
7     for  $j = 1$  to  $i - 1$ , in parallel :
8       if  $i \neq w$  and  $j \neq w$  then
9          $[[G_{i,j}]] \leftarrow [[G_{j,i}]]$ .
10 return  $[[D]] \leftarrow [[G]]$ .
```

Batch1, Batch2, and Batch3 are performed sequentially. Then the round complexity of the Floyd-Warshall protocol is reduced to $O(n)$.

Reduce secure operations. We further optimize the Floyd-Warshall protocol by reducing the number of minimum operations based on the characteristics of Isomap.

Optimization 1. In Isomap, all distances in the neighbor matrix are non-negative (specially, the distance from any sample to itself is zero) because of the non-negativity of distance metrics. We can conclude that the distance of the shortest path from any sample to itself is zero. Therefore,

- (1) The operations for updating the diagonal elements of the matrix, including the Batch1 and the operations of Batch3 at diagonal, can be saved because their values are known to be zero.
- (2) Batch2 can be saved because $G_{w',w'} = 0$ and the computation do not change the results.

Optimization 2. The output of the neighboring protocol is an undirected graph, meaning that the neighbor matrix is symmetric. The Floyd-Warshall algorithm obviously preserves the symmetry of the matrix, allowing us to eliminate the operations of Batch3 in lower (or upper) triangle. We can only perform the operations in the upper (or lower) triangle and copy the results to their corresponding transposed positions in the lower (or upper) triangle.

The final optimized protocol is presented in Protocol 4. For each $w \in \{1, \dots, n\}$, the protocol parallelly performs the operations of Batch3 in the upper triangle of the distance matrix, then copies the results to their corresponding transposed positions in the lower triangle. Compared to the plain Floyd-Warshall protocol, our optimized protocol reduces the number of secure operations from n^3 to $0.5n^3 - 1.5n^2 + n$.

4.3 Multidimensional Scaling

In the final step, Isomap uses multidimensional scaling (MDS) to construct the t -dimensional embedding of the dataset. On input a distance matrix $D \in \mathbb{R}^{n \times n}$ of all-pairs shortest paths, MDS computes the squared distance matrix, then applies double centering to convert the squared distance matrix into a matrix of inner products: $B = -\frac{1}{2}JD^2J$. The squared distance matrix is defined as $D^2 = \{D_{i,j}^2\}$.

Protocol 5: Secure multidimensional scaling

Notation: $[[Y]] \leftarrow \text{MDS}([D])$.
Input: Secret-shared distance matrix $D \in \mathbb{R}^{n \times n}$.
Output: Secret-shared embedding matrix $Y \in \mathbb{R}^{t \times t}$.

```

1 for  $i = 1$  to  $n - 1$ , in parallel :
2   for  $j = i + 1$  to  $n$ , in parallel :
3      $[[D_{i,j}]] \leftarrow [[D_{i,j}]]^2$ .
4      $[[D_{j,i}]] \leftarrow [[D_{i,j}]]$ .
5  $[[B]] \leftarrow -\frac{1}{2}J[[D]]J$ .
6  $[[v]], [[V]] \leftarrow \text{JACOBI}([B])$ .
7  $[[\pi]] \leftarrow \text{TOPK}([v], n - t)$ .
8 Apply  $[[\pi]]$  on  $[[v]]$  and the columns of  $[[V]]$  using
  permutation protocol.
9 Define  $[[\Lambda^t]]$  as a diagonal matrix whose diagonal
  elements are given by  $([[v_{n-t+1}]], \dots, [[v_n]])$ .
10 Define  $[[V^t]]$  as  $([[V_{i,j}]]_{1 \leq i \leq n, n-t+1 \leq j \leq n})$ .
11  $[[Y]] \leftarrow [[V^t]] [[\Lambda^t]]^{1/2}$ .
12 return  $[[Y]]$ .
```

J is the centering matrix defined as $\{J_{i,j} = I_{i,j} - \frac{1}{n}\}$ where I is an identity matrix. MDS performs eigen decomposition on B and computes the t -dimensional embedding as $Y = V^t \Lambda^t^{1/2}$, where Λ^t is the diagonal matrix formed by the t -largest eigenvalues and V^t is the matrix formed by the corresponding eigenvectors.

Secure MDS is shown in Protocol 5. We only perform element-square operations in the upper triangle of the distance matrix. We use the parallel Jacobi method to compute the eigenvalues and eigenvectors of the matrix. In our case, secure top- k arranges the k -smallest values to the first k positions of the vector. When we aim to find the t -largest eigenvalues, we can instead find the $(n-t)$ -smallest eigenvalues. Consequently, the eigenvalues in the last t positions of the vector will correspond to the t -largest eigenvalues.

We obtain SIsomap by putting it all together as in Appendix A.

5 Experiments

In this section, we demonstrate our improvements by evaluating (1) SIsomap's two building blocks, secure top- k and Floyd-Warshall protocols, and (2) the SIsomap system.

Implementation and setup. We implement SIsomap on top of a popular MPC framework, MP-SPDZ [25]. In all experiments, we set the ring size of the secret shares to 128 bits.

We run computing parties on three LEGION REN9000K-34IAZ machines. Each offers 12th Gen Intel(R) Core(TM) i9-12900KF 3.20 GHz CPU and 64GB RAM. These machines perform their computation on a single thread and are connected by a network with 5Gbps of bandwidth and 0.2ms of RTT. Our experiments measure the end-to-end performance because the semi-honest three-party replicated secret sharing scheme does not use a preprocessing paradigm.

5.1 Micro-Benchmarks

We independently evaluate the improvements of our top- k and all-pairs shortest paths methods. In addition to Isomap, they can also benefit many other privacy-preserving applications. In the micro-benchmarks, all input data are randomly generated. Note

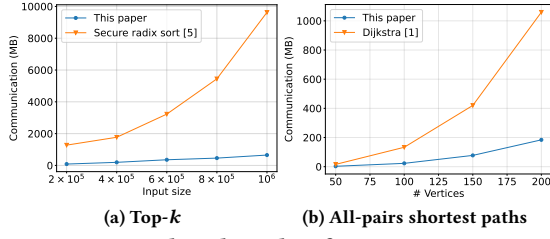


Figure 3: Micro-benchmarks of communication cost.

Table 3: Runtime (s) of top- k computation.

Input size	Secure radix sort [5]	This paper	Improvement
2×10^5	10.678	0.783	13.6×
4×10^5	21.869	2.029	10.8×
6×10^5	33.103	2.908	11.4×
8×10^5	43.808	4.86	9×
10^6	54.805	5.546	9.9×

Table 4: Runtime (s) of all-pairs shortest paths computation.

Number of vertices	Dijkstra [1]	Floyd-Warshall (This paper)	Improvement
50	65.749	0.096	684.9×
100	501.917	0.276	1818.5×
150	1634.98	1.156	1414.3×
200	3870.19	2.493	1552.4×
500	—	53.928	—
1000	—	438.748	—

that the distribution of data does not affect the computational or communication overhead, as all protocols are oblivious. Regardless of whether the input is real or random, the parties perform evaluations on secret shares, which are random by definition.

Top- k search. We compare our secure top- k protocol with the secure radix sort described in [5], which is considered the most practical solution in a three-party setting. Table 3 shows the runtime of the two methods where the number of input items ranges from 200,000 to 1,000,000. The secure top- k is 9 to 13.6 times faster than the secure radix sort. It can complete the top- k search for 200,000 secret-shared input items in less than 0.8 seconds, suggesting that it is nearly instantaneous for fewer input items. Figure 3a indicates that our protocol requires a significantly lower bandwidth.

All-pairs shortest paths computation. For a fair comparison, we implement the Dijkstra-based method [1] using the semi-honest replicated secret sharing and replace the shuffle protocol it uses with the efficient protocol [5] in this setting. As shown in Table 4, the proposed round-efficient Floyd-Warshall method significantly improves the efficiency of the computation of all pairs of shortest paths. Specifically, our Floyd-Warshall method is 684.9 to 1818.5 times faster than the Dijkstra-based method. This improvement mainly benefits from the reduction of communication overhead.

5.2 Privacy-Preserving Manifold Learning

Finally, we evaluate the performance of SIsomap on the Wisconsin breast cancer dataset from scikit-learn [44]. We assume that the

Table 5: SIsomap versus baseline method on runtime in seconds based on k -nearest neighbors.

Number of samples	Baseline	SIsomap	Improvement
50	75.646	6.787	11.1×
100	526.973	21.611	24.4×
200	4015.318	142.584	28.2×
300	—	619.984	—

Table 6: SIsomap versus baseline method on runtime in seconds based on radius neighbors.

Number of samples	Baseline	SIsomap	Improvement
50	74.641	6.634	11.3×
100	524.946	20.412	25.7×
200	4014.024	139.171	28.8×
300	—	589.238	—

dataset is distributed among multiple data owners. We reduce the original 30-dimensional dataset to a 5-dimensional embedding, and consider the k -nearest neighbors (k NN) and radius neighbors (RN) respectively. For the k NN-based protocol, we set the number of neighbors to 20. For the RN-based protocol, we set the radius to 2. The baseline for comparison uses the Dijkstra-based method to calculate the all-pairs shortest paths.

For the two neighboring approaches, SIsomap achieves a similar performance improvement over the baseline. The k NN-based SIsomap (resp. RN-based SIsomap) is about 11.1 to 28.2 times (resp. 11.3 to 28.8 times) faster than the baseline. As the number of samples increases, the performance improvements become more pronounced. The RN-based SIsomap is slightly faster than k NN-based SIsomap due to its more lightweight neighbor matrix computation, which does not require a top- k search.

6 Related Work

General-purpose MPC. General-purpose MPC protocols based on secret sharing [4, 10, 19, 22], garbled circuits [6, 34], and function secret sharing [7, 8] theoretically support arbitrary computations. However, directly applying them to complex tasks like Isomap often yields unsatisfactory performance. Efficient implementations require specialized optimizations for sub-routines such as top- k , shortest path, and eigen decomposition.

Secure dimensionality reduction. Existing works primarily focus on linear algorithms like PCA. For instance, PPCA [17] and other schemes [24] optimize eigen decomposition using Jacobi or QR methods under secret sharing. PrivateGraph [40] utilizes homomorphic encryption for spectral analysis but relies on a trusted third party. As these schemes mainly address covariance calculation and eigen decomposition, extending them to support manifold learning (particularly Isomap) remains challenging.

Acknowledgments

This work was supported by the Key Program of the National Natural Science Foundation of China under Grant U25B2028 and Grant 62432012.

References

- [1] Abdelrahman Aly and Sara Cleemput. 2022. A fast, practical and simple shortest path protocol for multiparty computation. In *European Symposium on Research in Computer Security*. 749–755.
- [2] Abdelrahman Aly, Edouard Cuvelier, Sophie Mawet, Olivier Pereira, and Mathieu Van Vyve. 2013. Securely solving simple combinatorial graph problems. In *Financial Cryptography and Data Security: 17th International Conference, FC 2013, Okinawa, Japan, April 1-5, 2013, Revised Selected Papers 17*. 239–257.
- [3] Abdelrahman Aly and Nigel P Smart. 2019. Benchmarking privacy preserving scientific operations. In *International Conference on Applied Cryptography and Network Security*. 509–529.
- [4] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. 2016. High-throughput semi-honest secure three-party computation with an honest majority. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 805–817.
- [5] Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, Katsumi Takahashi, and Junichi Tomida. 2022. Efficient secure three-party sorting with applications to data analysis and heavy hitters. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 125–138.
- [6] Gabrielle Beck, Aarushi Goel, Aditya Hegde, Abhishek Jain, Zhengzhong Jin, and Gabriel Kapthuk. 2023. Scalable multiparty garbling. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2158–2172.
- [7] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathi. 2021. Function secret sharing for mixed-mode and fixed-point secure computation. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. 871–900.
- [8] Elette Boyle, Niv Gilboa, and Yuval Ishai. 2019. Secure computation with preprocessing via function secret sharing. In *Theory of Cryptography: 17th International Conference, TCC 2019, Nuremberg, Germany, December 1–5, 2019, Proceedings, Part I 17*. 341–371.
- [9] Ran Canetti. 2001. Universally composable security: A new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*. 136–145.
- [10] Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. 2019. AS-TRA: high throughput 3pc over rings with application to secure prediction. In *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop*. 81–92.
- [11] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya Razenshteyn, and M Sadegh Riazi. 2020. {SANNs}: Scaling up secure approximate {k-Nearest} neighbors search. In *29th USENIX Security Symposium (USENIX Security 20)*. 2111–2128.
- [12] Heeyoul Choi and Seungjin Choi. 2007. Robust kernel isomap. *Pattern recognition* 40, 3 (2007), 853–862.
- [13] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. 2022. *Introduction to algorithms*.
- [14] Anders Dalskov, Daniel Escudero, and Marcel Keller. 2019. Secure evaluation of quantized neural networks. *arXiv preprint arXiv:1910.12435* (2019).
- [15] Muhammad Faisal, Jerry Zhang, John Liagouris, Vasiliki Kalavri, and Mayank Varia. 2023. {TVA}: A multi-party computation system for secure and expressive time series analytics. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5395–5412.
- [16] Brett Hemenway Falk and Rafail Ostrovsky. 2021. Secure Merge with $O(n \log \log n)$ Secure Operations. In *2nd Conference on Information-Theoretic Cryptography (ITC 2021)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 7–1.
- [17] Xiaoyu Fan, Guosai Wang, Kun Chen, Xu He, and Wei Xu. 2021. Ppca: Privacy-preserving principal component analysis using secure multiparty computation (mpc). *arXiv preprint arXiv:2105.07612* (2021).
- [18] Oded Goldreich. 2009. *Foundations of cryptography: volume 2, basic applications*.
- [19] Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. 2021. ATLAS: efficient and scalable MPC in the honest majority setting. In *Advances in Cryptology—CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II 41*. 244–274.
- [20] Koki Hamada, Ryo Kikuchi, Dai Ikarashi, Koji Chida, and Katsumi Takahashi. 2013. Practically efficient multi-party sorting protocols from comparison sort algorithms. In *Information Security and Cryptology—ICISC 2012: 15th International Conference, Seoul, Korea, November 28–30, 2012, Revised Selected Papers 15*. 202–216.
- [21] Feng Han, Lan Zhang, Hanwen Feng, Weiran Liu, and Xiangyang Li. 2022. Scape: Scalable collaborative analytics system on private database with malicious security. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 1740–1753.
- [22] Carmit Hazay, Emmanuela Orsini, Peter Scholl, and Eduardo Soria-Vazquez. 2022. TinyKeys: A new approach to efficient multi-party computation. *Journal of Cryptology* 35, 2 (2022), 3–33.
- [23] Chenfei Hu, Chuan Zhang, Dian Lei, Tong Wu, Ximeng Liu, and Liehuang Zhu. 2023. Achieving privacy-preserving and verifiable support vector machine training in the cloud. *IEEE Transactions on Information Forensics and Security* 18 (2023), 3476–3491.
- [24] Mehrad Jaberri and Hamid Mala. 2023. Privacy-preserving multi-party PCA computation on horizontally and vertically partitioned data based on outsourced QR decomposition. *The Journal of Supercomputing* 79, 13 (2023), 14358–14387.
- [25] Marcel Keller. 2020. MP-SPDZ: A versatile framework for multi-party computation. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 1575–1590.
- [26] Sven Laur, Jan Willemson, and Bingsheng Zhang. 2011. Round-efficient oblivious database manipulation. In *International Conference on Information Security*. Springer, 262–277.
- [27] Zhi Li, Hao Wang, Songnian Zhang, Wenying Zhang, and Rongxing Lu. 2023. SecKNN: FSS-based secure multi-party KNN classification under general distance functions. *IEEE Transactions on Information Forensics and Security* 19 (2023), 1326–1341.
- [28] Guopeng Lin, Weili Han, Wenqiang Ruan, Ruisheng Zhou, Lushan Song, Bing-shuai Li, and Yunfeng Shao. 2024. Ents: An Efficient Three-party Training Framework for Decision Trees by Communication Optimization. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 4376–4390.
- [29] Fengrun Liu, Xiang Xie, and Yu Yu. 2024. Scalable multi-party computation protocols for machine learning in the honest-majority setting. In *Proceedings of the 33rd USENIX Conference on Security Symposium (Philadelphia, PA, USA)*. Article 109, 18 pages.
- [30] Wen-jie Lu, Zhicong Huang, Qizhi Zhang, Yuchen Wang, and Cheng Hong. 2023. Squirrel: A Scalable Secure {Two-Party} Computation Framework for Training Gradient Boosting Decision Tree. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6435–6451.
- [31] Marina Meilă and Hanyu Zhang. 2024. Manifold learning: What, how, and why. *Annual Review of Statistics and Its Application* 11, 1 (2024), 393–417.
- [32] Payman Mohassel and Peter Rindal. 2018. ABY3: A mixed protocol framework for machine learning. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 35–52.
- [33] Payman Mohassel, Mike Rosulek, and Ni Trieu. 2020. Practical privacy-preserving k-means clustering. *Proceedings on privacy enhancing technologies* (2020), 414–433.
- [34] Payman Mohassel, Mike Rosulek, and Ye Zhang. 2015. Fast and secure three-party computation: The garbled circuit approach. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 591–602.
- [35] Duong Tung Nguyen and Ni Trieu. 2022. MPCache: privacy-preserving multi-party cooperative cache sharing at the edge. In *International Conference on Financial Cryptography and Data Security*. 80–99.
- [36] Benjamin Ostrovsky. 2024. Privacy-preserving dijkstra. In *Annual International Cryptology Conference*. 74–110.
- [37] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. 2024. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In *2024 IEEE Symposium on Security and Privacy (SP)*. 4753–4771.
- [38] Ashutosh Saxena, Abhinav Gupta, and Amitabha Mukerjee. 2004. Non-linear dimensionality reduction by locally linear isomaps. In *International Conference on Neural Information Processing*. 1038–1043.
- [39] Sacha Servan-Schreiber, Simon Langowski, and Srinivas Devadas. 2022. Private approximate nearest neighbor search with sublinear communication. In *2022 IEEE Symposium on Security and Privacy (SP)*. 911–929.
- [40] Sagar Sharma, James Powers, and Keke Chen. 2018. PrivateGraph: Privacy-preserving spectral analysis of encrypted graphs in the cloud. *IEEE Transactions on Knowledge and Data Engineering* 31, 5 (2018), 981–995.
- [41] Vin de Silva and Joshua B. Tenenbaum. 2002. Global versus local methods in nonlinear dimensionality reduction. In *Proceedings of the 16th International Conference on Neural Information Processing Systems*. 721–728.
- [42] Joshua B Tenenbaum, Vin de Silva, and John C Langford. 2000. A global geometric framework for nonlinear dimensionality reduction. *science* 290, 5500 (2000), 2319–2323.
- [43] Yongxin Tong, Xuchen Pan, Yuxiang Zeng, Yexuan Shi, Chunbo Xue, Zimu Zhou, Xiaofei Zhang, Lei Chen, Yi Xu, Ke Xu, et al. 2022. Hu-fu: Efficient and secure spatial queries over data federation. *Proceedings of the VLDB Endowment* 15, 6 (2022), 1159.
- [44] William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. 1993. Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5DW2B>.
- [45] Boshi Yuan, Shixuan Yang, Yongxiang Zhang, Ning Ding, Dawu Gu, and Shi-Feng Sun. 2024. {MD-ML}: Super Fast (Privacy-Preserving) Machine Learning for Malicious Security with a Dishonest Majority. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2227–2244.
- [46] Peng Zhang, Teng Huang, Xiaoqiang Sun, Wei Zhao, Hongwei Liu, Shangqi Lai, and Joseph K Liu. 2022. Privacy-preserving and outsourced multi-party k-means clustering based on multi-key fully homomorphic encryption. *IEEE Transactions on Dependable and Secure Computing* 20, 3 (2022), 2348–2359.
- [47] Wenting Zheng, Ryan Deng, Weikeng Chen, Raluca Ada Popa, Aurojit Panda, and Ion Stoica. 2021. Cerebro: A platform for {Multi-Party} cryptographic

Protocol 6: SIsomap**Notation:** $[[Y]] \leftarrow \text{SISOMAP}([X], k, d_{\text{radius}})$.**Input:** Secret-shared dataset $X \in \mathbb{R}^{n \times m}$ with n samples and m dimensions; the number of neighbors k and the radius d_{radius} .**Output:** Secret-shared embedding matrix $Y \in \mathbb{R}^{t \times t}$.

```

1 if  $k \neq \text{None}$  then
2    $[[G]] \leftarrow \text{KNEIGHBORS}([X], k)$ .
3 else
4    $[[G]] \leftarrow \text{RNEIGHBORS}([X], d_{\text{radius}})$ .
5  $[[D]] \leftarrow \text{FLOYD}([G])$ .
6  $[[Y]] \leftarrow \text{MDS}([D])$ .
7 return  $[[Y]]$ .

```

collaborative learning. In *30th USENIX Security Symposium (USENIX Security 21)*. 2723–2740.

- [48] Peizhao Zhou, Xiaojie Guo, Pinzhi Chen, Tong Li, Siyi Lv, and Zheli Liu. 2024. Shortcut: Making MPC-based Collaborative Analytics Efficient on Dynamic Databases. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 854–868.
- [49] Martin Zuber and Renaud Sirdey. 2021. Efficient homomorphic evaluation of k -NN classifiers. *Proceedings on Privacy Enhancing Technologies* (2021).

A Putting it all together: SIsomap

As shown in Protocol 6, SIsomap sequentially invokes Protocol 2 (or Protocol 3), Protocol 4 and Protocol 5 to compute the t -dimensional embedding $[[Y]]$ of input dataset $[[X]]$.

B Security Analysis

We use a standard simulation-based model to formalize the security of SIsomap. In the presence of a semi-honest adversary, this is modeled by showing that the real-world joint distribution of the views of corrupted parties and the outputs of honest parties is indistinguishable from its ideal-world counterpart, where the ideal-world views of corrupted parties are generated by a simulator given only the inputs and outputs of corrupted parties.

DEFINITION 1 ([18]). Let parties P_1, P_2, P_3 engage in protocol Π that computes deterministic functionality $f(in_1, in_2, in_3)$, where in_i denotes the input of party P_i . Let $\text{view}^\Pi(P_i)$ denote the view of party i during protocol Π , which consists of its input in_i , its internal random coins r_i , and messages m_i that were received by P_i in the execution. Let output^Π denote the joint output of all parties during protocol Π . We say that protocol Π securely computes functionality f in the presence of a semi-honest adversary that corrupts P_c , if there exists probabilistic polynomial time simulator $\mathcal{S}$ for every $c \in \{1, 2, 3\}$ such that (1) $f(in_1, in_2, in_3) = \text{output}^\Pi(in_1, in_2, in_3)$, and (2) $\{\mathcal{S}(c, in_c, f_c(in_1, in_2, in_3))\} \equiv \{\text{view}_c^\Pi(in_1, in_2, in_3)\}$ where $\equiv$ denotes indistinguishability.

We demonstrate the security of SIsomap within a hybrid model, where parties run a protocol with pairwise communication and also have access to a trusted party that computes a functionality for them. The sequential composition theorem from [9] states that security is preserved when such a trusted party is replaced by an MPC protocol

securely realizing this functionality. Given a functionality g , our protocol is said to work in the g -hybrid model.

SIsomap relies on a set of functionalities provided by MPC protocols, which have been proven secure in prior works [3, 5, 17, 32]: (1) $\mathcal{F}_{\text{sha}}$ and $\mathcal{F}_{\text{rev}}$ that are used for sharing data with parties and revealing the underlying data from shares, (2) numerical operations on secret shares, including multiplication $\mathcal{F}_{\text{mul}}$, comparison $\mathcal{F}_{\text{comp}}$, square root $\mathcal{F}_{\text{sqrt}}$, and matrix multiplication $\mathcal{F}_{\text{matmul}}$, (3) set operations on secret shares, including permutation $\mathcal{F}_{\text{permute}}$ and random shuffle $\mathcal{F}_{\text{shuffle}}$, (4) eigen decomposition on secret shares $\mathcal{F}_{\text{jacobi}}$.

Theorem 1 shows the security of Protocol 1. We identify $\mathcal{F}_{\text{topk}}$ as a deterministic functionality, as it essentially searches for the k -th element which is deterministic, regardless of the order of the remaining elements. We use $[[x]]_c$ to denote the secret shares of $[[x]]$ held by the corrupted party.

THEOREM 1. According to Definition 1, Protocol 1 securely computes functionality $\mathcal{F}_{\text{topk}}$ in the $\mathcal{F}_{\text{shuffle}}, \mathcal{F}_{\text{comp}}, \mathcal{F}_{\text{rev}}$ -hybrid model.

PROOF. The view of the adversary consists of appending bits $[[u]]_c$, appended input $[[b]]_c$, comparison result $[[e]]_c$, revealed comparison result e , pivot index p , partitioning range l, h , and random tapes. We construct the simulator $\mathcal{S}$ as follows.

$\mathcal{S}$ selects random tapes uniformly at random. Note that the subprotocols $\text{SHUFFLE}(\cdot)$, $[[\cdot]] < [[\cdot]]$, and $\text{REVEAL}(\cdot)$ are substituted with the invocation of subfunctionalities $\mathcal{F}_{\text{shuffle}}, \mathcal{F}_{\text{comp}}$, and $\mathcal{F}_{\text{rev}}$ executed by the trusted party, respectively.

$\mathcal{S}$ samples a uniformly random binary vector of the same length to simulate e , that is $e' \xleftarrow{\$} \{0, 1\}^{h-l+1}$ where the initial values of (l, h) is $(1, n)$. In each partitioning step of the real execution, the elements within the partitioning range are unique and are randomly shuffled. The calculation process of e is equivalent to randomly picking a pivot and comparing it with the remaining elements in a random order. Therefore, the comparison result e is evidently also a uniformly random binary vector, and it is perfectly simulated. Once e is perfectly simulated, $\mathcal{S}$ perfectly simulates p by setting $p' = \sum_{i=1}^n e'_i$. Values l, h are perfectly simulated by setting $l' = p' + 1$ if $p' < k$, and setting $h' = p' - 1$ otherwise.

As for secret shares $[[u]]_c$, $[[b]]_c$, and $[[e]]_c$, $\mathcal{S}$ uses uniformly random numbers to simulate them. These messages are perfectly simulated because of the definition of secret sharing scheme.

Thus, $\mathcal{S}$ perfectly simulates the view of the adversary.

The correctness of the protocol is immediate, as it only invokes the subprotocols that are substituted with the subfunctionalities in the hybrid model to implement the desired functionality $\mathcal{F}_{\text{topk}}$, and the vector shuffling does not affect the correctness. $\square$

The security of Protocols 2-6 is straightforward. These protocols invoke the functionalities of a hybrid model in a modular black-box fashion, with all intermediate values between the functionalities being secret-shared. Then the sequential composition theorem directly implies their security. The simulation in the hybrid model is plain, where the simulator calls the corresponding functionalities computed by the trusted party at each step and simulates all secret shares with uniformly random values.